Null Models and Sampling-Based Evaluation of Reinforcement Learning for LLVM Pass Ordering

Dimitrije Pešić
School of Electrical Engineering, University of Belgrade
Belgrade, Serbia
pd230014d@student.etf.bg.ac.rs

***Abstract*—Reinforcement learning (RL) for compiler pass ordering is usually evaluated by one deterministic rollout of the learned policy, compared against a fixed optimization level such as `-O3`. We show that both conventions can reverse conclusions. We define two null models, a single random episode and best-of-N random search, in the same curated 36-pass action space the policies act in, and evaluate policies both by a deterministic rollout and by best-of-k sampling. Across 880 programs from eleven independent sources, random search in the curated space alone beats `-Oz` on all nine real-code sources (0.9–24.5%). A graph-neural policy trained on only four small programs beats the budget-matched random null on 33 of 36 suite–seed totals, significant within ten of eleven sources (one-sided Wilcoxon); random search needs 1.4–6$\times$ the compilation budget, or more, to close that margin. Linux kernel code, where random search alone already suffices, is where the margin ends. The same policy matches greedy search at 73% of its compilation budget in all three seeds; an identically trained flat-feature policy does so in one seed of three. Best-of-8 sequences from the curated space, learned or random, yield `.text` sections 10.4–10.6% (x86) and 15% (Cortex-M) smaller than `-Oz` and apply faster than the `-Oz` pipeline. Finally, RL gradients alone leave the graph policy's deterministic mode on a plateau below `-Oz`; distilling hand-crafted features into the encoder before RL lifts it off. In this setting, evaluation choices changed the conclusions more than the choice of representation did.**

***Index Terms*—compiler optimization, LLVM, phase ordering, reinforcement learning, graph neural networks, evaluation methodology**

I. Introduction

The order in which a compiler applies its optimization passes affects the quality of the generated code. Choosing a good order per program, the *phase-ordering problem*, is undecidable in general [1], and exhaustive enumeration is feasible only with heavy pruning [2]. Fixed pipelines such as LLVM's `-O3` and `-Oz` apply one hand-tuned sequence to every program; a line of work therefore applies deep reinforcement learning to choose program-specific orderings [3], [4], [5], [6].

Reported results, however, rest on two rarely questioned evaluation conventions. First, learned policies are compared against fixed optimization levels (often `-O3`, a *speed*-oriented pipeline, even when the objective is code *size*) rather than against a null model that measures how much of the gain needs no learning. Second, policies are evaluated by a single deterministic (argmax) rollout, that is, by the mode of the policy distribution rather than by the distribution itself.

This paper asks what remains of an RL pass-ordering system once both conventions are replaced. We train proximal policy optimization (PPO) [7] agents with two state representations, the flat 56-dimensional Autophase feature vector [3] and a custom GraphSAGE [8] encoder over control- and data-flow graphs parsed from LLVM IR, under a controlled protocol in which the representation is the only variable. We evaluate them on 880 programs from eleven independent sources against null models defined in the same action space with the same step budget.

Our contributions are threefold. (1) The curated 36-pass action space is itself the dominant effect: best-of-50 random search inside it comes within 0.5% of greedy search on cBench and beats `-Oz` on every real-code suite, a fact any evaluation should control for. (2) The evaluation mode decides the ranking of representations: under argmax the two agents are statistically indistinguishable, whereas under best-of-k sampling the graph policy beats the paired null on 33 of 36 suite–seed totals (24 of 24 on the original seven sources) and matches greedy-search quality at 73% of its budget in every seed, which the flat policy does in one seed of three. (3) RL gradients alone leave the graph policy's deterministic mode on a plateau; distilling Autophase features into the encoder before RL lifts it off and improves deterministic-rollout generalization by 9–10%. All results are reproducible from a public repository with pinned dependencies.[1]

II. Related Work

AutoPhase [3] introduced deep RL over a flat feature vector for phase ordering; CompilerGym [4] standardized the environment we build on. ProGraML [9] showed that graph representations of programs outperform flat features on supervised compiler tasks. POSET-RL [5] and the coreset approach of Liang et al. [6] reduce the action space before learning; the latter, closest to ours, selects from a 50-sequence coreset with a graph neural network (GNN) and reports an average 4.7% gain over `-Oz`. Our study is complementary: using null models inside the same reduced space, a control rarely reported, it measures *where such gains come from* and shows that the decoding mode changes conclusions. Cummins et al. [4] report that plain random search is competitive with several autotuners on cBench; we formalize that observation

[1] `https://github.com/dimitrijepesic/compiler-opt`

into a null-model protocol and replicate it across nine real-code sources. Mammadli et al. [10] train RL over `-O3` subsequences and find it inferior to `-O3` on held-out programs, consistent with our argmax findings. Large language models have also been applied, at a far larger budget [11].

III. Experimental Protocol

Environment and metric. We use CompilerGym 0.2.5 (`llvm-ic-v0`, LLVM 10) [4]. The optimization objective is LLVM IR instruction count (IC); Section IV-E validates IC against actual binary section sizes. `-O3`/`-Oz` baselines are the environment's real optimization-level observations, not approximated pass lists.

Action space. Profiling all 124 available passes on the 14-program cBench training split yields 36 passes that each improve at least one program; the other 88 improve none. All agents and null models operate in this space with 45-step episodes. The reference greedy search tries all 124 passes at every step and stops when none improves (≈1 970 compilations per cBench program).

Agents and training. Both agents use PPO (clip 0.2, γ=0.99, generalized advantage estimation with λ=0.95) with truncation-aware advantage bootstrapping, early stopping of update epochs on the Kullback–Leibler divergence (target 0.02), and a linearly decaying entropy coefficient. The Autophase agent observes a $\log(1+x)$-transformed 56-dimensional feature vector; the GNN agent parses the IR into a graph (one-hot opcode plus four scalar features per instruction; separate control-flow and data-flow edge types) and encodes it with a three-layer edge-type-aware GraphSAGE network. Both train on the *same* four small programs of that split (<3000 instructions), for the same 100 000 environment steps, with three seeds each; checkpoints are selected on a three-program validation subset. Apart from the joint optimizer that the encoder shared by the policy and value heads requires (value-loss weight 0.5, encoder learning rate 10^{-4}), the representation is the only variable.

Null models. Two null models are measured in the same 36-pass space: (a) a single random 45-step episode (the null for one policy rollout) and (b) the best of N random episodes (the null for search). When a policy is evaluated by best-of-k sampling, its null is the best of the *same number* k of stored random episodes on the same program, paired by construction.

Evaluation. Policies are evaluated in two modes: one deterministic argmax rollout, and best-of-k stochastic rollouts sampled from the policy distribution (GNN rollouts sample with the encoder's dropout active). The battery spans eleven sources: cBench (val+test, 9), MiBench [12] (40), CHStone [13] (12), NPB (122), BLAS (50), a 97-program POJ-104 sample, 150-program samples of the AnghaBench, GitHub and Linux-kernel datasets shipped with CompilerGym [4], and two synthetic generators (csmith [14], 50; llvm-stress, 50): 871 programs plus cBench, none seen in training. Policy evaluation is restricted to programs up to 6000 instructions (831 of the 871 eligible; eight fail feature extraction, leaving 823) to bound rollout cost. The four added sources (AnghaBench, GitHub, Linux, llvm-stress) store 16 samples per program: the first eight for comparability, the disjoint second eight as a built-in resampling check.

TABLE I
Total IR instruction count per suite: real optimization levels vs. best-of-50 random episodes in the curated 36-pass space (ΔOz > 0: random below `-Oz`).

Suite (n)	-O0	-O3	-Oz	Rnd-50	ΔOz
AnghaBench (150)	8 902	4 193	4 043	**4 006**	+0.9%
GitHub (150)	224 490	222 369	224 171	**221 297**	+1.3%
MiBench (40)	12 270	6 317	4 841	**4 765**	+1.6%
Linux (150)	255 140	250 862	252 031	**246 983**	+2.0%
BLAS (50)	40 745	39 141	37 838	**36 757**	+2.9%
cBench (9)	232 442	163 834	115 987	**111 006**	+4.3%
CHStone (12)	19 857	14 182	9 834	**9 097**	+7.5%
POJ-104 (97)	21 099	11 083	8 334	**7 622**	+8.5%
NPB (122)	130 327	70 659	59 542	**44 981**	+24.5%
csmith (50)	352 963	82 950	**57 625**	60 626	−5.2%
llvm-stress (50)	5 739	238	**234**	238	−1.7%

Encoder pretraining. In one arm (two seeds), the GNN encoder is first trained to regress the Autophase vector from the program graph (2430 states from random episodes on nine training-split programs of up to 15 000 instructions, so this arm also sees more programs than the four-program RL stage) and then fine-tuned with RL.

IV. Results

A. The curated action space, not learning, is the main effect

Table I compares real optimization levels against best-of-50 random episodes in the 36-pass space. Random search beats `-Oz` on all nine real-code sources, by 0.9% (AnghaBench) to 24.5% (NPB); on NPB even *single* random episodes sum to 11.5% below `-Oz`, a total driven by the largest programs (only 32% of individual episodes win). The margins are thinnest on AnghaBench, GitHub and Linux (0.9–2.0%), whose code largely arrives pre-optimized (on GitHub `-Oz` removes only 0.14% of `-O0` IC; on GitHub and Linux, `-O3` beats `-Oz`). `-O3` is the wrong yardstick for size (on 6% of the programs under 500 instructions it *increases* IC above `-O0`). On the synthetic generators the full `-Oz` pipeline keeps its advantage (csmith 5.2%, llvm-stress 1.7%): the curated space was distilled from real code and transfers to real code only.

B. Sampling separates the representations

Under argmax, both policies transfer poorly beyond the training set (on NPB the GNN's deterministic mode degenerates to 79 686 total IC vs. `-Oz` 53 085), and the two representations are statistically indistinguishable. Table II reports best-of-8 sampled rollouts against the paired best-of-8 random null: the GNN policy beats its null on 33 of 36 suite–seed totals across the eleven sources, 24 of 24 on the original seven. Because seeds within a suite share the stored null episodes, the primary test uses the twelve independent suite/split units (cBench counted as two): the GNN wins the same ten under every seed (exact sign test p=0.019; the original eight alone give p=3.9×10^{-3}). One-sided Wilcoxon tests on the median seed are significant within ten of eleven sources (largest

TABLE II

BEST-OF-8 SAMPLED ROLLOUTS (PER-CELL MEDIAN OVER THREE SEEDS) VS. THE PAIRED BEST-OF-8 RANDOM NULL AND `-Oz`. AP: AUTOPHASE. ISO-k: MEDIAN RANDOM EPISODES MATCHING THE GNN BEST-OF-8; SAFE%: MEAN PER-PROGRAM GAIN OF $\min(\text{BEST-OF-8}, \texttt{-Oz})$ OVER `-Oz` (MEDIAN SEED). ADDED SOURCES: FIRST 8 OF 16 STORED SAMPLES; n: PROGRAMS WHERE ALL SEEDS SUCCEEDED.

Suite (n)	Null	-Oz	AP	GNN	iso-k	safe%
MiBench (40)	4 828	4 841	4 783	**4 782**	23	1.8
CHStone (12)	9 241	9 834	9 133	**9 120**	32	7.3
BLAS (50)	37 099	37 838	36 998	**36 847**	20	2.0
csmith (28)	17 555	20 226	17 179	**17 064**	>50	18.2
NPB (120)	40 301	53 085	40 822	**39 909**	11	10.0
POJ-104 (97)	7 853	8 334	7 917	**7 730**	12	7.9
AnghaB. (150)	4 049	4 043	4 052	**4 027**	17	1.1
GitHub (140)	79 713	80 661	79 738	**79 676**	17	1.2
Linux (136)	**147 216**	148 029	147 278	147 378	2	0.6
llvm-str. (50)	255	**234**	245	240	49	3.1

p=0.025, MiBench). The exception is Linux kernel code, where best-of-50 random search already beats `-Oz` by 2% and the policy adds nothing (per-program $p \geq 0.24$ in two of three seeds; zero of three second-slice wins). On GitHub one seed loses the summed IC by 17 while the per-program test is significant under every seed ($p \leq 2\times10^{-3}$). The identically trained Autophase policy wins 21 of 36 suite–seed totals and is significant on five of eleven sources. The margin over the null is modest (0.7–2.8% of suite totals on the original battery) but systematic, and it holds on the synthetic csmith programs, where best-of-50 random search *loses* to `-Oz` on the full set of 50 (Table I): on the 28 evaluated programs the sampled GNN policy is 2.8% below its null and 16% below `-Oz`. As search cost, the same margin is large. On eight of the ten sources where the GNN wins, random search needs a median of 11–49 episodes (1.4–6× the budget) to match its eight samples; on cBench and csmith, the 50 stored episodes match no seed and one of three seeds respectively. On Linux, where the policy does not win, two random episodes suffice (Table II, iso-k). Per program on the original battery, the GNN sampler is at least as good as its null on 88–92% of the 347 programs (175W/145T/27L, seed 42). Because the $(k{+}1)$-th candidate can be `-Oz` itself, the deployed sampler is never worse than `-Oz` by construction and averages 0.6–18.2% per-program savings (Table II, safe%).

Robustness. A k=32 rerun of the original suites reproduces the suite-total win everywhere except NPB, where only one seed of three reproduces it and the total flips in 7 of 12 disjoint eight-sample slices; the per-program Wilcoxon on NPB nevertheless stays significant in every resampling ($p<0.05$ in all twelve slices, $p<10^{-3}$ in each rerun's first eight). Where totals are dominated by a few large programs (NPB, GitHub, Linux), per-suite tests, not win counts, are the stronger evidence.

Fig. 1 places these results on a quality-vs.-budget front across the original battery: matching the GNN's best-of-8 takes random search ≈13 episodes per program, and random search converges to the GNN curve at 32–33 episodes. On cBench, against measured greedy search, the sampled GNN policy at k=8 (360 compilations per program) is within 0.4% of greedy (0.06–0.35% across seeds); at k=32 (73% of greedy's

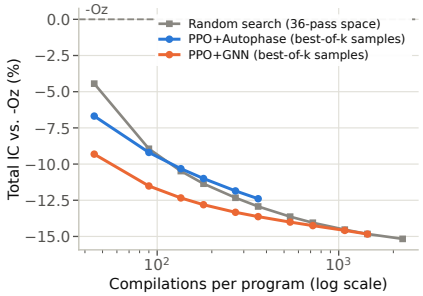

Fig. 1. Quality vs. budget across the 347 programs of the original battery, summed IC relative to `-Oz`: best-of-k sampling (GNN k=1..32, Autophase k=1..8, mean over three seeds) vs. expected best-of-k random search.

budget) it reaches greedy quality, nominally surpassing it in two of three seeds (110 494 and 110 527 vs. 110 550; the third 110 787), while all three seeds beat the paired null (111 023). Autophase at k=32 surpasses greedy in one seed (110 338) but trails the null in the other two (111 056, 111 108): the graph policy's advantage is consistency, not peak quality. Within the measured budgets, random search never catches the sampler on cBench; on the original battery it does so at the larger budget (k=32), first on NPB, the suite where random search is strongest. The learned sampler thus reduces the cost of search where search is most expensive.

C. Controls: where the sampling advantage lives

Four controls on the same 347 programs and nulls (seed 42; Table III) decompose the margin. An *untrained* policy with identical architecture and sampling protocol is statistically indistinguishable from random search on every suite (all one-sided $p \geq 0.12$) and loses five of six suite totals: the advantage is learned. Disabling dropout reproduces the result on all six suites (largest p=0.016) and is indistinguishable from dropout-active sampling (58W/226T/63L): the exploration comes from the policy distribution itself. An *open-loop* variant samples all 45 actions from $\pi(\cdot \mid s_0)$, so best-of-8 costs one graph pass plus eight sequence applications (≈0.13 s per module). It reaches 90–141% of the closed-loop gain on MiBench, BLAS and CHStone but loses it on NPB and POJ-104 (0.35 of the gain overall): the learned per-program *prior* suffices on simpler suites; state feedback pays off on harder ones. Finally, a fixed eight-sequence *portfolio* mined by greedy set cover from random search on the 14 training programs (no model at inference) beats the null on five of six suites and the learned sampler on MiBench, BLAS and CHStone (one-sided $p \leq 0.006$). The policy keeps better totals on csmith, NPB and POJ-104, significantly so on csmith (one-sided p=0.004 on per-program medians across seeds). Much of the sampler's value on small, regular programs is thus distillable into a static sequence list, consistent with coreset-style approaches [6], while per-program adaptation pays off where random search is weakest.

D. Pretraining unlocks the encoder

All three from-scratch GNN seeds reach the same validation plateau (summed IC 689 on the three-program subset) within

TABLE III
CONTROLS (SEED 42, 347 PROGRAMS): SUMMED IC OF BEST-OF-8 VARIANTS VS. THE STORED BEST-OF-8 NULL (116 877); WINS: SUITES (OF 6) BEATING IT. PORTFOLIO-16 VS. THE BEST-OF-16 NULL (115 158).

Variant	Wins	Total IC
Untrained policy, best-of-8	1/6	116 941
Open-loop $\pi(\cdot\|s_0)$, best-of-8	4/6	116 377
Portfolio-8 (no model)	5/6	115 972
GNN best-of-8 (no dropout: 115 761)	6/6	115 451
Portfolio-16 (no model, 2× budget)	6/6	114 407

the first 10–17% of the budget and never leave it, *worse* than `-Oz` (643) and a single random episode (640): RL gradients alone do not lift the encoder's deterministic mode off the plateau. Distilling Autophase into the encoder before RL leaves the plateau in two of two seeds (best validation 651, 668) and improves the mean full-split argmax totals from 64 550/69 180 to 57 980/63 229 (validation/test), by 10.2% and 8.6%, the first deterministic policy here to beat the single-episode random null on both splits. Pretraining repairs the policy's *mode*; whether it also improves the policy as a *sampler* was not measured.

E. Binary-level metrics

IC is a proxy, so we compiled the optimized modules with the environment's own LLVM 10 code generator and measured section sizes. On programs with at least 1000 instructions, relative IC reduction strongly predicts relative `.text` reduction (Pearson r=0.78, p=2.2×10^{-6}; n=26 points, `-Oz` and the best random episode for each of 13 programs); below that size, fixed code-generation overheads dominate and the proxy weakens (r=0.19 over all 36 programs). The IC gains translate into bytes: across the 347 paired programs of the original battery, best-of-8 GNN sampling (seed 123) produces `.text` sections 10.6% smaller than `-Oz` (864 887 vs. 967 878 bytes; a fresh best-of-8 random draw achieves 10.4%). On the 24 larger programs (above 6000 instructions) best-of-8 random search is 12–29% *above* `-Oz` in two independent draws, so over all 371 programs it lands between 4.0% below and 0.1% above. The 45-pass sequences also apply *faster* than the `-Oz` pipeline (median 15.0–15.8 ms vs. 19.5 ms per module). Cross-compiling for a Cortex-M target (`thumbv7m`; an approximation, as the IR keeps the x86 host datalayout) makes the savings *larger*: 14.9–15.0% below `-Oz` (839 vs. 986 kB), with sampler and random null equal, so on this target the curated space delivers the entire margin. Binary footprint is >99.9% data/BSS, invariant under ordering (<0.03%); `.text` is the informative metric.

V. DISCUSSION AND LIMITATIONS

Three practices follow. Evaluations of learned pass ordering should (1) report a random null inside the same action space and budget; (2) report sampling-based as well as deterministic evaluation, since the two can change conclusions; and (3) never use `-O3` as a size baseline. Limitations: the study optimizes IR instruction count, whose correlation with binary sections is validated only above 1000 instructions. Training used four small programs by design (the experimental control); a three-seed arm trained on nine programs of up to 15 000 instructions did not improve argmax generalization on average (64 797/69 748 vs. 64 550/69 180; one seed improved both splits, two regressed). Runtime is not measured. The stack is CompilerGym 0.2.5 (LLVM 10), though the mined portfolio ported to LLVM 18's new pass manager still beats its `-Oz` by 10.6–10.9% in summed `.text` (best of 8 or 16 sequences by measured `.text`) over the 314 of 319 programs from five suites that port (169–174 per-program wins vs. 83–89 losses, p<10^{-11}; csmith excluded, 20 of 28 fail to port). The GNN costs ∼19× the flat agent's cost per training step, though open-loop deployment cuts inference to one graph pass.

VI. CONCLUSION

Under controlled measurement, the value of RL for LLVM pass ordering shifts: most of the headline gain over `-Oz` belongs to the curated action space. What survives the controls is the learned graph policy's role as an *amortized searcher*: greedy-search quality at 73% of the compilation budget, transferable from four small programs to ten of eleven program sources. Its deterministic mode becomes usable only after a short pretraining step.

ACKNOWLEDGMENT

The author thanks Prof. Zaharije Radivojević for early discussions of this work.

REFERENCES

[1] S.-A.-A. Touati and D. Barthou, "On the decidability of phase ordering problem in optimizing compilation," in *Proc. ACM Computing Frontiers*, 2006.

[2] P. A. Kulkarni, D. B. Whalley, G. S. Tyson, and J. W. Davidson, "Exhaustive optimization phase order space exploration," in *Proc. IEEE/ACM CGO*, 2006.

[3] A. Haj-Ali *et al.*, "AutoPhase: Juggling HLS phase orderings in random forests with deep reinforcement learning," in *Proc. MLSys*, 2020.

[4] C. Cummins *et al.*, "CompilerGym: Robust, performant compiler optimization environments for AI research," in *Proc. IEEE/ACM CGO*, 2022.

[5] S. Jain, Y. Andaluri, S. VenkataKeerthy, and R. Upadrasta, "POSET-RL: Phase ordering for optimizing size and execution time using reinforcement learning," in *Proc. IEEE ISPASS*, 2022.

[6] Y. Liang *et al.*, "Learning compiler pass orders using coreset and normalized value prediction," in *Proc. ICML*, 2023.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.

[8] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. NeurIPS*, 2017.

[9] C. Cummins, Z. Fisches, T. Ben-Nun, T. Hoefler, M. O'Boyle, and H. Leather, "ProGraML: A graph-based program representation for data flow analysis and compiler optimizations," in *Proc. ICML*, 2021.

[10] R. Mammadli, A. Jannesari, and F. Wolf, "Static neural compiler optimization via deep reinforcement learning," in *Proc. IEEE/ACM LLVM-HPC Workshop*, 2020.

[11] C. Cummins *et al.*, "LLM compiler: Foundation language models for compiler optimization," in *Proc. ACM CC*, 2025.

[12] M. R. Guthaus, J. S. Ringenberg, D. Ernst, T. M. Austin, T. Mudge, and R. B. Brown, "MiBench: A free, commercially representative embedded benchmark suite," in *Proc. IEEE WWC*, 2001.

[13] Y. Hara, H. Tomiyama, S. Honda, H. Takada, and K. Ishii, "CHStone: A benchmark program suite for practical C-based high-level synthesis," in *Proc. IEEE ISCAS*, 2008.

[14] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in *Proc. ACM PLDI*, 2011.